

Dimensionality Reduction

Dimensionality Reduction

What is Dimensionality Reduction?

- Dimensionality reduction is the process of **reducing the number of input features (variables)** in a dataset while **preserving as much information** as possible.
- In practice, it's about **simplifying** the data: fewer inputs, less noise, and more tractable modeling and visualization.
- A classic example is keeping either **weight in kilograms** or **weight in pounds** (not both), because they encode the same information.

Why is Dimensionality Reduction Required?

- **Storage & speed:** Fewer columns → smaller files and **faster training/inference**.
- **Model performance:** Some algorithms degrade in high-D spaces (distance metrics become less informative; linear models suffer from multicollinearity).
- **Overfitting control:** Removing noisy/irrelevant features improves generalization.
- **Visualization:** Reducing to **2D/3D** enables pattern discovery and sanity checks.
- **Redundancy removal:** Collapses correlated/duplicate signals (e.g., treadmill time vs. calories).

Two Main Approaches

A) Feature Selection (keep a subset of original features)

You **do not transform** features; you **choose** which to keep. It's attractive when interpretability matters because selected features retain their original meaning.

Families

- **Filter methods:** Purely statistical/heuristic criteria; **model-agnostic**.
- **Wrapper methods:** Search subsets and **evaluate with a model**.
- **Embedded methods:** The model **learns which features matter** during training (e.g., LASSO, trees).

B) Feature Extraction (create new features)

You **transform** the data into a new, usually smaller set of features (components, factors, embeddings). These are often **less interpretable** but can capture structure the raw features miss.

Families

- **Linear:** PCA, LDA, ICA, Factor Analysis, SVD.
- **Non-linear/manifold:** Isomap, LLE, t-SNE, UMAP.

Neural: Autoencoders (including denoising and variational variants)

1) Missing Value Ratio (Filter)

What: Drop columns with too many missing values (set a threshold, e.g., 20–50%).

Why: Columns dominated by NaNs rarely add signal.

Tips:

- Decide threshold by **domain knowledge** and **imputation feasibility**.
- If a variable is critical, consider **target-aware imputation** instead of dropping.

2) Low Variance Filter (Filter)

What: Remove features with near-zero variance.

Why: If a column barely changes, it can't explain target variation.

Tips:

- **Scale** first if mixing different units.
- For classification, consider **variance within class** as well.

3) High Correlation Filter (Filter)

What: Identify feature pairs with $|\rho|$ above a threshold (e.g., 0.8) and drop one.

Why: Reduces **multicollinearity** and stabilizes linear models.

Tips:

- Use **Pearson** for linear, **Spearman** for monotonic relations.
- Keep the one that's **more correlated with the target** or more interpretable.

4) Random Forest Feature Importance (Embedded)

What: Train a tree ensemble and rank features by importance.

Why: Trees capture **non-linearities** and **interactions**.

Gotchas:

Impurity-based importances can be biased toward **high-cardinality** features.

Prefer **permutation importance** as a robustness check.

5) Backward Feature Elimination (Wrapper)

What: Start with **all** features; iteratively remove the least useful.

Why: Directly optimizes performance with your chosen model/metric.

Costs: Computationally heavy (retrain many times).

Good for: Small-to-medium feature sets where accuracy is paramount.

6) Forward Feature Selection (Wrapper)

What: Start with **none**; add features one by one that most improve performance.

Why: Greedy but practical search for a compact subset.

Costs/Risks: Still expensive; can get **stuck** in local optima.

7) Factor Analysis (Feature Extraction — Linear)

Goal: Model observed variables as combinations of a smaller number of **latent factors**.

Assumptions: Linear structure, factors explain shared variance, **unique variance** per variable.

Steps:

Check sampling adequacy (**KMO**) and **Bartlett's test**.

Choose number of factors (eigenvalues > 1 , scree, parallel analysis).

Rotate (Varimax/Promax) for interpretability.

Use when: You expect **conceptual groupings** (e.g., socioeconomic, psychometrics).

8) Principal Component Analysis — PCA (Feature Extraction — Linear)

Goal: Orthogonal components that capture maximal variance.

How:

Center/scale $\rightarrow$ compute covariance (or use **SVD**).

Eigenvectors = component directions; eigenvalues = explained variance.

Choosing k: Scree elbow, cumulative explained variance (e.g., 90–95%).

Tips:

Standardize if features have different scales.

Interpretation: Components are **linear combinations** (not original features).

Where it shines: Compression, denoising, speed-ups for distance-based methods.

9) Independent Component Analysis — ICA (Feature Extraction — Linear)

Goal: Recover **statistically independent**, non-Gaussian sources from observed mixtures.

Contrast with PCA: PCA $\Rightarrow$ **uncorrelated**; ICA $\Rightarrow$ **independent** (stronger).

Common algorithm: **FastICA** (maximizes non-Gaussianity via kurtosis/negentropy).

Use cases: Signal separation (EEG/ICA), audio “cocktail party” problems.

10) Linear Discriminant Analysis — LDA (Feature Extraction — Supervised)

Goal: Find projections that **maximize class separation** (between-class / within-class).

Assumptions: Class-conditional normality, shared covariance (for LDA classifier).

Use: Dimensionality reduction **for classification** (great as a preprocessor).

Limit: Max components = **#classes - 1**.

11) t-Distributed Stochastic Neighbor Embedding — t-SNE (Manifold/Non-linear)

Goal: Preserve **local neighborhoods** in 2D/3D for visualization.

Key params: **Perplexity** (neighborhood size), **learning rate**, **iterations**.

Strengths: Reveals clusters beautifully.

Limits:

Non-parametric (no transform for new points unless you fit again).

Poor at preserving **global** structure; distances across clusters can mislead.

Sensitive to hyperparameters and scaling.

12) Autoencoders (Neural Feature Extraction)

What: Neural nets learn a low-D **bottleneck** that reconstructs inputs.

Variants: **Denoising** (robustness), **Sparse** (sparsity penalties), **Variational** (probabilistic latent space).

Pros: Non-linear, scalable to images/text/audio.

Cons: Needs tuning, less interpretable; risk of learning **identity** without care (use regularization/undercomplete bottleneck).

Also commonly used:

SVD / TruncatedSVD: Matrix factorization; TruncatedSVD works well on **sparse** data (e.g., text).

Isomap: Preserves **geodesic** distances on a manifold (graph shortest paths).

LLE: Preserves local linear reconstruction; good for unfolding manifolds.

UMAP: Graph-based manifold learning; often **faster** than t-SNE and preserves **more global** structure.

Key params: n_neighbors, min_dist.

Principal Component analysis

- Principal Component Analysis (PCA) is a **linear dimensionality reduction** technique that can be utilized for extracting information from a high-dimensional space by projecting it into a lower-dimensional sub-space. It tries to preserve the essential parts that have more variation of the data and remove the non-essential parts with fewer variation.
- it is an **Unsupervised** dimensionality reduction technique, you can cluster the similar data points based on the feature correlation between them without any supervision (or labels)

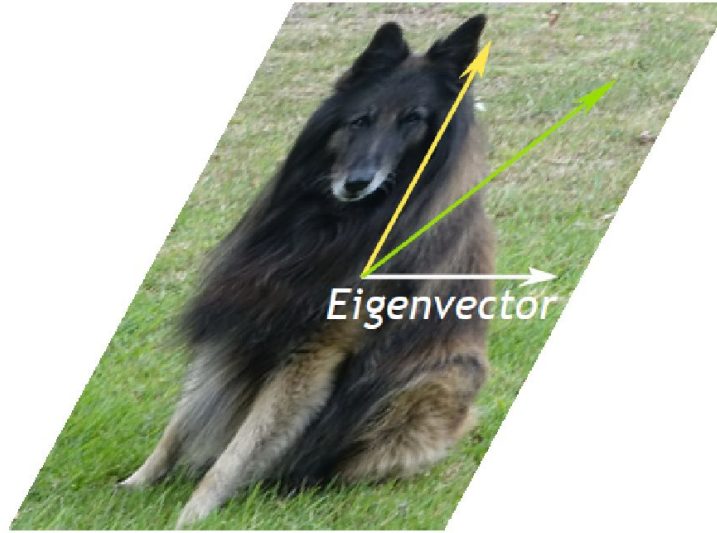
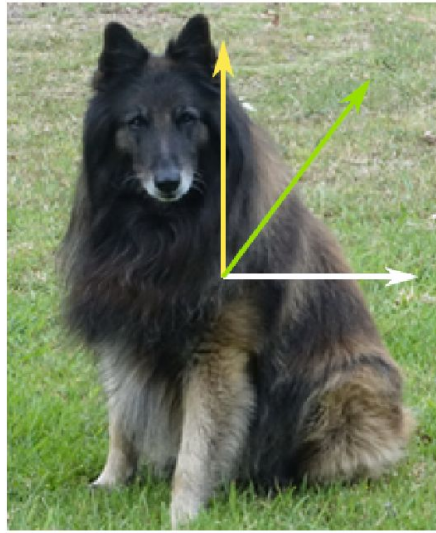
Uses of PCA:

- It is used to find inter-relation between variables in the data.
- It is used to interpret and visualize data.
- As number of variables are decreasing it makes further analysis simpler.
- It's often used to visualize genetic distance and relatedness between populations.

- Principal component analysis (PCA) is a statistical procedure to describe a set of multivariate data of possibly correlated variables by relatively few numbers of linearly uncorrelated variables.
- The uncorrelated variables are created as linear combination of the original variables and in the decreasing order of importance so that the first component explains most of the original variation in the data.
- Each component is orthogonal to the other component to explain the variation that is not already explained by other components.
- So in layman's term, if a dataset is explained by several different characteristics, PCA summarizes those characteristics to define fewer characteristics to explain the data well. Moreover, it does remove the redundancy or correlation that can exist when explaining data using large number of variables and create a list of uncorrelated variables.

Eigenvector and Eigen value

A simple example is that an eigenvector **does not change direction** in a transformation:



Principal Component analysis

For a square matrix **A**, an Eigenvector and Eigenvalue make this equation true:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

Diagram illustrating the equation $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. Arrows point from the labels below to the terms in the equation: "Matrix" points to $\mathbf{A}$, "Eigenvector" points to $\mathbf{v}$ in $\mathbf{A}\mathbf{v}$, and "Eigenvalue" points to λ in $\lambda\mathbf{v}$.

Example: For this matrix

$$\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix}$$

an eigenvector is:

$$\begin{bmatrix} 1 \\ 4 \end{bmatrix}$$

with a matching eigenvalue of 6

Let's do some matrix multiplies to see what we get.

$\mathbf{A}\mathbf{v}$ gives us:

$$\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} -6 \times 1 + 3 \times 4 \\ 4 \times 1 + 5 \times 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 24 \end{bmatrix}$$

$\lambda\mathbf{v}$ gives us :

$$6 \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 24 \end{bmatrix}$$

Yes they are equal! So $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ as promised.

How do we find these eigen things?

We start by finding the **eigenvalue**: we know this equation must be true:

$$Av = \lambda v$$

Now let us put in an identity matrix so we are dealing with matrix-vs-matrix:

$$Av = \lambda Iv$$

Bring all to left hand side:

$$Av - \lambda Iv = 0$$

If v is non-zero then we can solve for λ using just the determinant:

$$| A - \lambda I | = 0$$

Let's try that equation on our previous example:

Example: Solve for λ :

Start with $|A - \lambda I| = 0$

$$\left| \begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right| = 0$$

Which is:

$$\begin{vmatrix} -6-\lambda & 3 \\ 4 & 5-\lambda \end{vmatrix} = 0$$

Calculating that determinant gets:

$$(-6-\lambda)(5-\lambda) - 3 \times 4 = 0$$

Which then gets us this Quadratic Equation:

$$\lambda^2 + \lambda - 42 = 0$$

And solving it gets:

$$\lambda = -7 \text{ or } 6$$

And yes, there are **two** possible eigenvalues.

Now we know **eigenvalues**, let us find their matching **eigenvectors**.

Example (continued): Find the Eigenvector for the Eigenvalue $\lambda = 6$:

Start with:

$$Av = \lambda v$$

Put in the values we know:

$$\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 6 \begin{bmatrix} x \\ y \end{bmatrix}$$

After multiplying we get these two equations:

$$-6x + 3y = 6x$$

$$4x + 5y = 6y$$

Bringing all to left hand side:

$$-12x + 3y = 0$$

$$4x - 1y = 0$$

Either equation reveals that $\mathbf{y} = 4\mathbf{x}$, so the **eigenvector** is any non-zero multiple of th

$$\begin{bmatrix} 1 \\ 4 \end{bmatrix}$$

And we get the solution shown at the top of the page:

$$\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} -6 \times 1 + 3 \times 4 \\ 4 \times 1 + 5 \times 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 24 \end{bmatrix}$$

... and also ...

$$6 \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 24 \end{bmatrix}$$

So $\mathbf{Av} = \lambda \mathbf{v}$

Why?

What is the purpose of these?

One of the cool things is we can use matrices to do transformations in space, which is used a lot in computer graphics.

In that case the eigenvector is "the direction that doesn't change direction" !

And the eigenvalue is the scale of the stretch:

- **1** means no change,
- **2** means doubling in length,
- **-1** means pointing backwards along the eigenvalue's direction

There are also many applications in physics, etc.

Example: find the eigenvalues for this 3x3 matrix:

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 4 & 5 \\ 0 & 4 & 3 \end{bmatrix}$$

First calculate $A - \lambda I$:

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 4 & 5 \\ 0 & 4 & 3 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2-\lambda & 0 & 0 \\ 0 & 4-\lambda & 5 \\ 0 & 4 & 3-\lambda \end{bmatrix}$$

Now the determinant should equal zero:

$$\begin{vmatrix} 2-\lambda & 0 & 0 \\ 0 & 4-\lambda & 5 \\ 0 & 4 & 3-\lambda \end{vmatrix} = 0$$

Which is:

$$(2-\lambda) [(4-\lambda)(3-\lambda) - 5 \times 4] = 0$$

This ends up being a cubic equation, but just looking at it here we see one of the roots is **2** (because of $2-\lambda$), and the part inside the square brackets is Quadratic, with roots of **-1** and **8**.

So the Eigenvalues are **-1**, **2** and **8**

Example (continued): find the Eigenvector that matches the Eigenvalue **-1**

Put in the values we know:

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 4 & 5 \\ 0 & 4 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = -1 \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

After multiplying we get these equations:

$$\begin{aligned} 2x &= -x \\ 4y + 5z &= -y \\ 4y + 3z &= -z \end{aligned}$$

Bringing all to left hand side:

$$\begin{aligned} 3x &= 0 \\ 5y + 5z &= 0 \\ 4y + 4z &= 0 \end{aligned}$$

So **x = 0**, and **y = -z** and so the **eigenvector** is any non-zero multiple of this:

$$\begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix}$$

TEST Av :

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 4 & 5 \\ 0 & 4 & 3 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 4-5 \\ 4-3 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}$$

And λv :

$$-1 \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}$$

So $Av = \lambda v$, yay!

(You can try your hand at the eigenvalues of **2** and **8**)

STEP BY STEP EXPLANATION OF PCA

STEP 1: STANDARDIZATION

The aim of this step is to standardize the range of the continuous initial variables so that each one of them contributes equally to the analysis.

More specifically, the reason why it is critical to **perform standardization prior to PCA**, is that the latter is quite sensitive regarding the variances of the initial variables. **That is, if there are large differences between the ranges of initial variables, those variables with larger ranges will dominate over those with small ranges** (For example, a variable that ranges between 0 and 100 will dominate over a variable that ranges between 0 and 1), which will lead to biased results. So, transforming the data to comparable scales can prevent this problem.

Mathematically, this can be done by subtracting the mean and dividing by the standard deviation for each value of each variable.

$$z = \frac{\text{value} - \text{mean}}{\text{standard deviation}}$$

STEP 2: COVARIANCE MATRIX COMPUTATION

The aim of this step is to understand how the variables of the input data set are varying from the mean with respect to each other, or in other words, to see if there is any relationship between them. Because sometimes, variables are highly correlated in such a way that they contain redundant information. **So, in order to identify these correlations, we compute the covariance matrix.**

The covariance matrix is a $p \times p$ symmetric matrix (where p is the number of dimensions) that has as entries the covariances associated with all possible pairs of the initial variables. For example, for a 3-dimensional data set with 3 variables x , y , and z , the covariance matrix is a 3×3 matrix of this form:

$$\begin{bmatrix} Cov(x, x) & Cov(x, y) & Cov(x, z) \\ Cov(y, x) & Cov(y, y) & Cov(y, z) \\ Cov(z, x) & Cov(z, y) & Cov(z, z) \end{bmatrix}$$

What do the covariances that we have as entries of the matrix tell us about the correlations between the variables?

It's actually the sign of the covariance that matters :

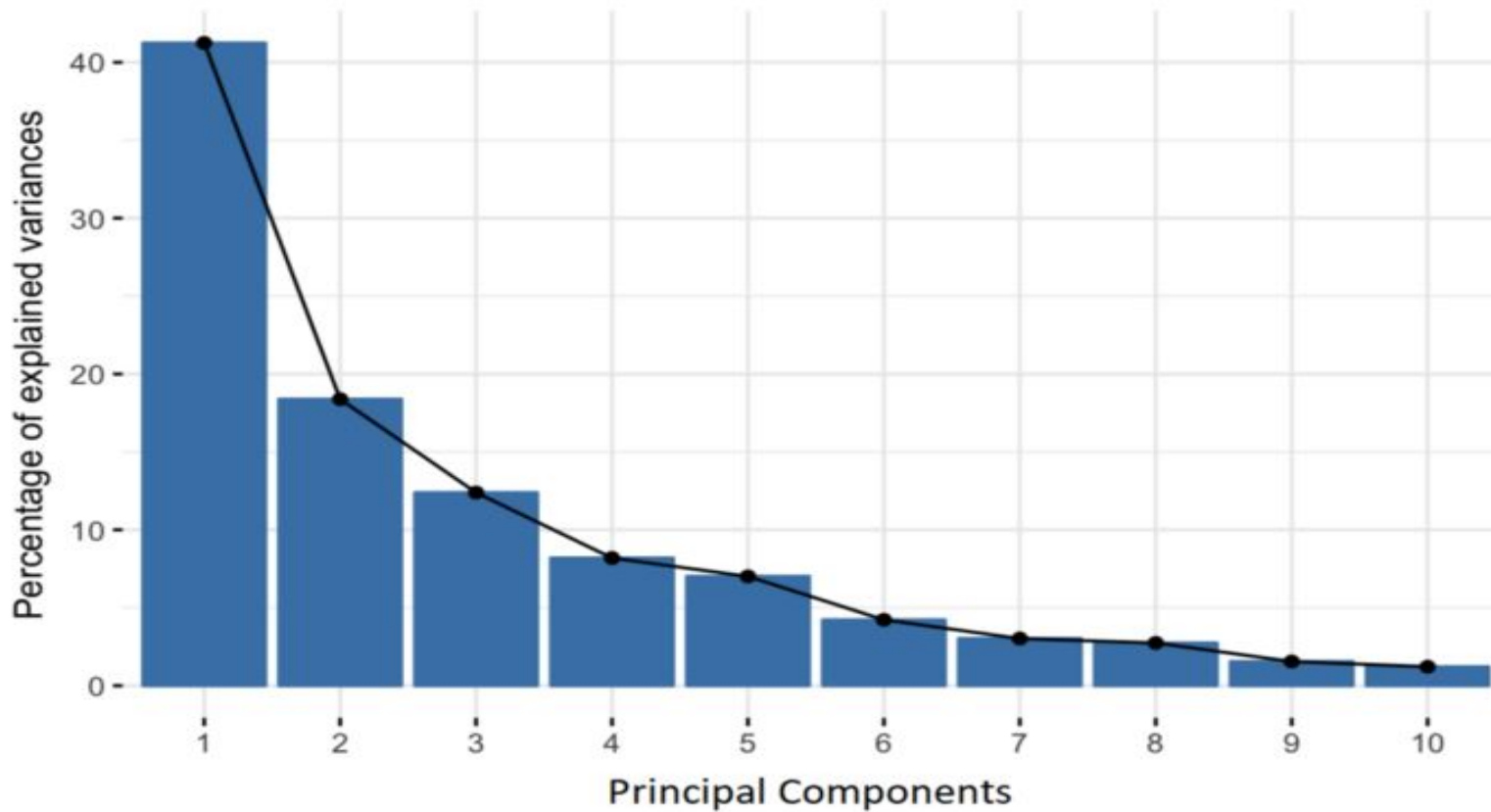
- if positive then : the two variables increase or decrease together (correlated)
- if negative then : One increases when the other decreases (Inversely correlated)

Now, that we know that the covariance matrix is not more than a table that summaries the correlations between all the possible pairs of variables, let's move to the next step.

STEP 3: COMPUTE THE EIGENVECTORS AND EIGENVALUES OF THE COVARIANCE MATRIX TO IDENTIFY THE PRINCIPAL COMPONENTS

Eigenvectors and eigenvalues are the linear algebra concepts that we need to compute from the covariance matrix in order to determine the **principal components of the data**. Before getting to the explanation of these concepts, let's first understand what do we mean by principal components.

Principal components are new variables that are constructed as linear combinations or mixtures of the initial variables. These combinations are done in such a way that the new variables (i.e., principal components) are uncorrelated and most of the information within the initial variables is squeezed or compressed into the first components. So, the idea is 10-dimensional data gives you 10 principal components, but PCA tries to put maximum possible information in the first component, then maximum remaining information in the second and so on, until having something like shown in the scree plot below.



Geometrically speaking, principal components represent the directions of the data that explain a maximal amount of variance, that is to say, the lines that capture most information of the data. The relationship between variance and information here, is that, the larger the variance carried by a line, the larger the dispersion of the data points along it, and the larger the dispersion along a line, the more the information it has. To put all this simply, just think of principal components as new axes that provide the best angle to see and evaluate the data, so that the differences between the observations are better visible.

HOW PCA CONSTRUCTS THE PRINCIPAL COMPONENTS

As there are as many principal components as there are variables in the data, principal components are constructed in such a manner that the first principal component accounts for the **largest possible variance** in the data set.

For example, let's assume that the scatter plot of our data set is as shown below, can we guess the first principal component? Yes, it's approximately the line that matches the purple marks because it goes through the origin and **it's the line in which the projection of the points (red dots) is the most spread out**. Or **mathematically speaking, it's the line that maximizes the variance** (the average of the squared distances from the projected points (red dots) to the origin).

